

Dr. H N National College of Engineering

*(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi and Govt. of Karnataka)
36B Cross, Jayanagar 7th block, Bengaluru – 560070*

Department of ISE



Fourth Year

A Mini-Project Report On

“BANKING MANAGEMENT SYSTEM”

*Submitted in the fulfilment of the requirements for
ADVANCED JAVA (BIS402)*

SUBMITTED BY :

JEEVITHA . L
MANOGNASHREE KJ

1HC24IS014
1HC24IS022

IVth SEMESTER

Under the Guidance of

Dr Nandita Y
Assistant Professor

DEPT. OF INFORMATION SCIENCE AND ENGINEERING

Dr. H N National College of Engineering

*(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi and Govt. of Karnataka)
36B Cross, Jayanagar 7th block, Bengaluru – 560070*

**Department of ISE
2025-2026****Certificate**

This is to certify that the Mini Project Work entitled “**BANKING MANAGEMENT SYSTEM**” is a bonafide work carried out by **JEEVITHA.L (1HC24IS014), MANOGNASHREE KJ (1HC24IS022)** in partial fulfilment for the IV semester of Bachelor of Engineering in ISE of the Visvesvaraya Technological University, Belgaum during the year 2025-2026. The Project report has been approved as it satisfies the academics prescribed for the Bachelor of Engineering degree.

Signature of Guide
Dr Nandita Y

Signature of HOD
Dr Purushotham.U

TABLE OF CONTENTS

CHAPTERS:-

PAGE NO:-

1.INTRODUCTION

4

2.PROBLEM DESCRIPTION

6

3. HIGH LEVEL DESIGN

7

4. CODE

10

5. RESULT

30

6.CONCLUSION

31

CHAPTER -1**INTRODUCTION****WHAT IS BANKING MANAGEMENT SYSTEM ?**

A Banking Management System (BMS) is specialized software that automates and manages core banking operations such as customer accounts, transactions, loans, and reporting, ensuring efficiency, security, and compliance. It is widely used by banks to streamline services, reduce manual work, and provide customers with faster, more reliable access to financial services.

** Key Features of a Banking Management System**

- Account Management: Handles savings, current, and fixed deposit accounts.
- Transaction Management: Automates deposits, withdrawals, fund transfers, and transaction history.
- Customer Relationship: Stores customer details and supports services like loan applications.
- Loan Management: Manages loan requests, approvals, EMI calculations, and repayments.
- Security: Implements encryption, fraud detection, and multi-factor authentication.
- Reporting: Generates financial statements and audit-ready records.
- Integration: Connects with ATMs, payment gateways, and external banking networks.

Advantages of Banking Management System

- Automation of Banking Operations Reduces manual work by handling deposits, withdrawals, loans, and reporting automatically.
- Database Connectivity (JDBC) Ensures secure and efficient storage/retrieval of customer and transaction data.
- Scalability with Servlets & JSP Can handle multiple users simultaneously with session management and request handling.
- Security Features Advanced Java supports authentication, encryption, and role-based access control.

- Cross-Platform Support Java applications run on any OS with JVM, making deployment easier.
- User-Friendly Interfaces Swing/JavaFX or JSP provide interactive GUIs for customers and staff.
- Integration Capabilities Can connect with ATMs, payment gateways, and external APIs using Java libraries.
- Maintainability & Modularity Object-oriented design allows modular classes (Account, Customer, Transaction), making updates easier
- Real-Time Processing Transactions are updated instantly in the database, ensuring accuracy.

CHAPTER-2**PROBLEM DESCRIPTION**

The Banking Management System implemented in Advanced Java is a comprehensive application designed to simulate and automate real-world banking operations. It addresses the challenges of manual banking such as inefficiency, errors, and lack of security by providing a structured, GUI-based solution. The system allows banks to register customers, create accounts of different types (Savings, Current, Fixed Deposit, Salary, Joint, Student), and perform essential operations like deposits, withdrawals, transfers, and balance checks. It also supports advanced features such as loan applications, EMI calculations with automatic deductions from account balances, interest addition, transaction history tracking, and generation of mini statements.

The application is built using object-oriented design principles, with dedicated classes for *Customer*, *Account*, *Transaction*, and *Loan*, ensuring modularity and maintainability. The *BankUI* class provides an interactive Swing interface with gradient panels, rounded buttons, and an output console for real-time feedback. Each operation is validated to prevent errors such as duplicate IDs, invalid inputs, or insufficient balances, thereby ensuring data integrity. Transactions are recorded with timestamps, and reports can be generated for auditing and customer reference.

In essence, this system solves the problem of manual banking inefficiency by integrating multiple banking functions into a single cohesive platform. It combines security, automation, scalability, and user-friendliness, making it suitable both as an academic project and as a prototype for real banking applications. It demonstrates how Advanced Java technologies (Swing, event handling, collections, and OOP) can be used to build a robust financial management system that mirrors real banking workflows.

CHAPTER-3

HIGH LEVEL DESIGN** High-Level Design of Banking Management System****1. Presentation Layer (User Interface)**

- **Technology:** Java Swing (JFrame, JPanel, JTextField, JButton, JTextArea, etc.)
- **Responsibilities:**
 - Provides a graphical interface for users to interact with the system.
 - Displays forms for entering Customer ID, Account Number, Account Type, and Amount.
 - Offers operation buttons (Login, Add Customer, Create Account, Deposit, Withdraw, Transfer, Loan, EMI, Reports, etc.).
 - Shows results and feedback in the **Output Console**.

2. Business Logic Layer

- **Core Classes:**
 - **Customer** → Stores customer details (ID, name, address, phone, Aadhaar).
 - **Account** → Manages account details, deposits, withdrawals, and interest.
 - **Transaction** → Records transaction details (type, amount, date).
 - **Loan** → Handles loan applications, EMI calculation, and repayment status.
- **Responsibilities:**
 - Implements rules for account creation, deposits, withdrawals, transfers.
 - Validates inputs (e.g., sufficient balance, duplicate IDs).
 - Calculates EMI and deducts payments from account balance.
 - Updates transaction history and loan status.

3. Data Management Layer

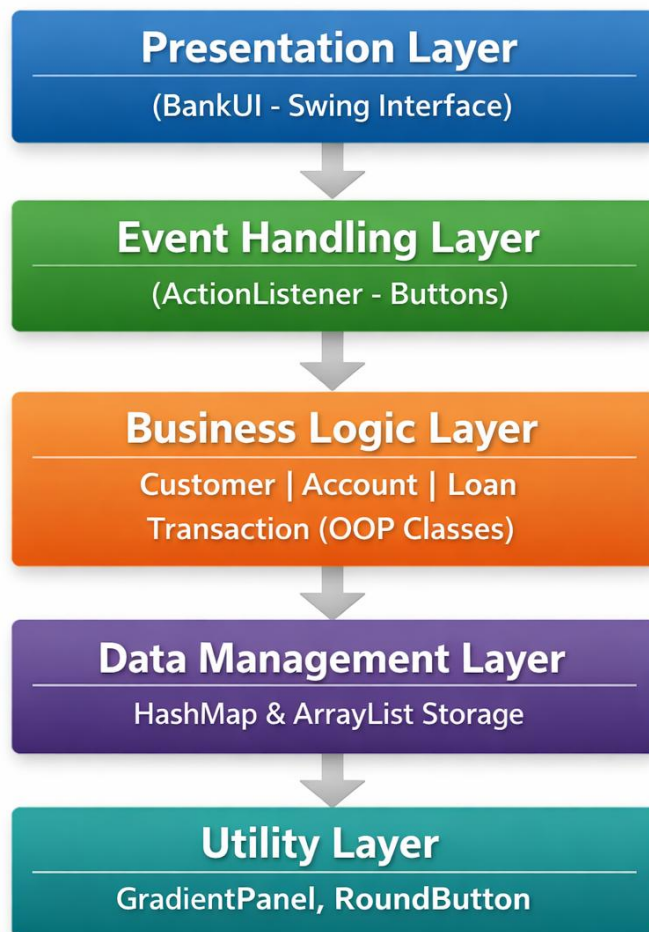
- **Storage:** In-memory collections (HashMap, ArrayList).
 - Map<String, Customer> → Stores customers.
 - Map<String, Account> → Stores accounts.
 - ArrayList<Transaction> → Stores transactions.
 - ArrayList<Loan> → Stores loans.
- **Responsibilities:**
 - Maintains all records during program execution.
 - Provides quick lookup for accounts and customers.
 - Stores transaction logs for reporting and mini statements.

4. Event Handling Layer

- **Technology:** ActionListener (event-driven programming).
- **Responsibilities:**
 - Captures button clicks and triggers corresponding operations.
 - Validates user inputs before calling business logic

5. Utility Layer

- **Components:**
 - **GradientPanel** → Custom UI panel with gradient background.
 - **RoundButton** → Custom button with hover effects.
 - **SimpleDateFormat** → Formats transaction timestamps.
- **Responsibilities:**
 - Enhances UI aesthetics and usability.
 - Provides reusable components for consistent design.

 High-Level Architecture Diagram (Textual)

CHAPTER-4**CODE**

```
package Banking;

import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import java.awt.event.*;
import java.awt.geom.*;
import java.text.SimpleDateFormat;
import java.util.*;

/* =====

ADVANCED BANK MANAGEMENT SYSTEM – Fixed & Beautified

Bugs Fixed:

1. Broken string literal "\n" inside emiBtn output text
2. Missing transferBtn actionPerformed handler
3. EMI payment now deducts from account balance
4. Loan months input validated via dialog

===== */

class Customer {
    String customerId, name, address, phone, aadhaar;

    public Customer(String customerId, String name, String address,
                    String phone, String aadhaar) {
        this.customerId = customerId;
        this.name      = name;
        this.address   = address;
        this.phone     = phone;
        this.aadhaar   = aadhaar;
    }
}
```

```
    }  
}  
class Account {  
    String accountNumber, customerId, accountType;  
    double balance;  
    public Account(String accountNumber, String customerId,  
        String accountType, double openingBalance) {  
        this.accountNumber = accountNumber;  
        this.customerId = customerId;  
        this.accountType = accountType;  
        this.balance = openingBalance;  
    }  
    public void deposit(double amount) { balance += amount;  
}  
    public boolean withdraw(double amount) {  
        if (amount > balance) return false;  
        balance -= amount;  
        return true;  
    }  
    public void addInterest() { balance += balance * 0.04; }  
}  
class Transaction {  
    String accountNumber, type, date;  
    double amount;  
    public Transaction(String accountNumber, String type,  
        double amount, String date) {  
        this.accountNumber = accountNumber;  
        this.type = type;  
        this.amount = amount;
```

```
        this.date = date;
    }
}

class Loan {
    String loanId, accountNumber, status;
    double amount, emi, paidAmount;
    int months;

    public Loan(String loanId, String accountNumber,
        double amount, int months) {
        this.loanId = loanId;
        this.accountNumber = accountNumber;
        this.amount = amount;
        this.months = months;
        this.status = "Approved";
        this.emi = Math.round((amount / months) * 100.0) / 100.0;
        this.paidAmount = 0;
    }

    public boolean payEMI(Account acc) {
        if (paidAmount >= amount) return false;
        if (!acc.withdraw(emi)) return false;    // BUG FIX #3: deduct from account
        paidAmount += emi;
        if (paidAmount >= amount) {
            paidAmount = amount;
            status = "Completed";
        }
        return true;
    }
}

}class GradientPanel extends JPanel {
    private final Color c1, c2;
```

```
public GradientPanel(Color c1, Color c2) {
    this.c1 = c1; this.c2 = c2;
    setOpaque(false);
}

@Override protected void paintComponent(Graphics g) {
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON);
    g2.setPaint(new GradientPaint(0, 0, c1, getWidth(), getHeight(), c2));
    g2.fillRect(0, 0, getWidth(), getHeight());
    super.paintComponent(g);
}

}class RoundButton extends JButton {
private Color base, hover, current;

public RoundButton(String text, Color base, Color hover) {
    super(text);
    this.base = base; this.hover = hover; this.current = base;
    setOpaque(false); setContentAreaFilled(false); setBorderPainted(false);
    setFocusPainted(false);
    setForeground(Color.WHITE);
    setFont(new Font("Segoe UI", Font.BOLD, 13));
    setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    addMouseListener(new MouseAdapter() {
        public void mouseEntered(MouseEvent e) { current = hover; repaint(); }
        public void mouseExited (MouseEvent e) { current = base; repaint(); }
    });
}

@Override protected void paintComponent(Graphics g) {
    Graphics2D g2 = (Graphics2D) g.create();
```

```
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        g2.setColor(current);

        g2.fillRect(0, 0, getWidth(), getHeight(), 14, 14);

        g2.setColor(new Color(0,0,0,30));

        g2.drawRoundRect(0, 0, getWidth()-1, getHeight()-1, 14, 14);

        g2.dispose();

        super.paintComponent(g);
    }
}
```

```
public class BankUI extends JFrame implements ActionListener {
```

```
    static final Color BG_TOP   = new Color(10, 25, 60);
    static final Color BG_BOT   = new Color(20, 50, 100);
    static final Color PANEL_BG = new Color(255, 255, 255, 22);
    static final Color ACCENT    = new Color(0, 198, 255);
    static final Color ACCENT2   = new Color(0, 255, 180);
    static final Color BTN_RED   = new Color(220, 60, 60);
    static final Color BTN_RED2  = new Color(255, 90, 90);
    static final Color BTN_GRN   = new Color(30, 180, 100);
    static final Color BTN_GRN2  = new Color(50, 220, 130);
    static final Color BTN_BLU   = new Color(40, 120, 220);
    static final Color BTN_BLU2  = new Color(70, 160, 255);
    static final Color BTN_PRP   = new Color(130, 60, 200);
    static final Color BTN_PRP2  = new Color(170, 100, 255);
    static final Color BTN_ORG   = new Color(220, 130, 0);
    static final Color BTN_ORG2  = new Color(255, 170, 30);
    static final Color BTN_TEL   = new Color(0, 160, 160);
    static final Color BTN_TEL2  = new Color(0, 210, 210);
    static final Color BTN_DARK  = new Color(50, 50, 80);
```

```
static final Color BTN_DARK2 = new Color(80, 80, 120);

JTextField customerField, accountField, amountField;
JComboBox<String> accountTypeBox;
JTextArea outputArea;
JButton loginBtn, addCustomerBtn, createAccountBtn, depositBtn,
        withdrawBtn, transferBtn, balanceBtn, transactionBtn,
        miniStatementBtn, loanBtn, emiBtn, reportBtn, interestBtn, clearBtn;

Map<String, Customer> customers  = new HashMap<>();
Map<String, Account>  accounts   = new HashMap<>();
ArrayList<Transaction> transactions = new ArrayList<>();
ArrayList<Loan>       loans      = new ArrayList<>();
SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy HH:mm:ss");
public BankUI() {
    setTitle("Advanced Bank Management System");
    setSize(1200, 720);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setUndecorated(false);

    GradientPanel root = new GradientPanel(BG_TOP, BG_BOT);
    root.setLayout(null);
    setContentPane(root);

    JLabel icon = new JLabel("🏦");
    icon.setFont(new Font("Segoe UI Emoji", Font.PLAIN, 36));
    icon.setBounds(40, 20, 50, 50);
    root.add(icon);

    JLabel title = new JLabel("BANK MANAGEMENT SYSTEM");
    title.setFont(new Font("Georgia", Font.BOLD, 30));
```

```
title.setForeground(Color.WHITE);
title.setBounds(100, 18, 700, 50);
root.add(title);

JLabel sub = new JLabel("Secure · Smart · Seamless");
sub.setFont(new Font("Segoe UI", Font.ITALIC, 14));
sub.setForeground(ACCENT);
sub.setBounds(104, 52, 400, 22);
root.add(sub);

JPanel formPanel = makeGlassPanel(20, 85, 480, 290);
root.add(formPanel);

addFormRow(formPanel, "Customer ID",  customerField = makeField(), 15, 15);
addFormRow(formPanel, "Account No.",  accountField = makeField(), 15, 70);
addFormRow(formPanel, "Account Type",  null, 15, 125);
accountTypeBox = new JComboBox<>(new String[] {
    "Select Account Type", "Savings Account", "Current Account",
    "Fixed Deposit Account", "Salary Account", "Joint Account", "Student Account"});
styleCombo(accountTypeBox);
accountTypeBox.setBounds(185, 125, 270, 36);
formPanel.add(accountTypeBox);

addFormRow(formPanel, "Amount (₹)",  amountField = makeField(), 15, 180);
addFormRow(formPanel, "",  null, 15, 235);

JPanel btnPanel = makeGlassPanel(20, 390, 700, 290);
root.add(btnPanel);

JLabel btnTitle = new JLabel("Operations");
btnTitle.setFont(new Font("Segoe UI", Font.BOLD, 15));
btnTitle.setForeground(new Color(180, 220, 255));
btnTitle.setBounds(15, 8, 200, 25);
btnPanel.add(btnTitle);
```



```
loginBtn    = addBtn(btnPanel, "🔑 Login", BTN_BLU, BTN_BLU2, 15, 42);
addCustomerBtn = addBtn(btnPanel, "👤 Add Customer", BTN_GRN, BTN_GRN2,
235, 42);

createAccountBtn= addBtn(btnPanel, "🏧 Create Account", BTN_PRP, BTN_PRP2,
455, 42);

depositBtn    = addBtn(btnPanel, "💰 Deposit",    BTN_GRN, BTN_GRN2, 15,
100);

withdrawBtn    = addBtn(btnPanel, "💸 Withdraw",    BTN_RED, BTN_RED2,
235, 100);

transferBtn    = addBtn(btnPanel, "🔄 Transfer",    BTN_BLU, BTN_BLU2, 455,
100);

balanceBtn    = addBtn(btnPanel, "📊 Check Balance", BTN_TEL, BTN_TEL2,
15, 158);

transactionBtn = addBtn(btnPanel, "📋 Transactions", BTN_ORG, BTN_ORG2,
235, 158);

miniStatementBtn= addBtn(btnPanel, "📄 Mini Statement", BTN_BLU, BTN_BLU2,
455, 158);

loanBtn        = addBtn(btnPanel, "🏠 Apply Loan",   BTN_PRP, BTN_PRP2, 15,
216);

emiBtn         = addBtn(btnPanel, "📅 Pay EMI",      BTN_ORG, BTN_ORG2, 235,
216);

reportBtn      = addBtn(btnPanel, "📈 Reports",     BTN_TEL, BTN_TEL2, 455,
216);

interestBtn    = addBtn(btnPanel, "💵 Add Interest", BTN_GRN, BTN_GRN2, 15,
258);

clearBtn       = addBtn(btnPanel, "🗑 Clear",        BTN_DARK, BTN_DARK2, 235,
258);

JPanel outPanel = makeGlassPanel(510, 85, 560, 595);

root.add(outPanel);

JLabel outTitle = new JLabel("Output Console");

outTitle.setFont(new Font("Segoe UI", Font.BOLD, 15));

outTitle.setForeground(ACCENT2);
```

```
outTitle.setBounds(15, 10, 250, 25);
outPanel.add(outTitle);

outputArea = new JTextArea();
outputArea.setEditable(false);
outputArea.setOpaque(false);
outputArea.setFont(new Font("JetBrains Mono", Font.PLAIN, 14));
outputArea.setForeground(Color.YELLOW);
outputArea.setCaretColor(ACCENT);
outputArea.setBorder(BorderFactory.createEmptyBorder(5, 10, 5, 5));
JScrollPane scroll = new JScrollPane(outputArea);
scroll.setBounds(10, 40, 535, 540);
scroll.setOpaque(false);
scroll.getViewport().setOpaque(false);
scroll.setBorder(BorderFactory.createLineBorder(new Color(255,255,255,40), 1));
outPanel.add(scroll);

JLabel status = new JLabel(" System ready • All data in-memory");
status.setFont(new Font("Segoe UI", Font.PLAIN, 12));
status.setForeground(new Color(150,200,255));
status.setBounds(20, 688, 600, 22);
root.add(status);

setVisible(true);
printWelcome();
}

private JPanel makeGlassPanel(int x, int y, int w, int h) {
    JPanel p = new JPanel(null) {
        @Override protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
```

```
g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

g2.setColor(new Color(255,255,255,20));

g2.fillRoundRect(0, 0, getWidth(), getHeight(), 20, 20);

g2.setColor(new Color(255,255,255,50));

g2.drawRoundRect(0, 0, getWidth()-1, getHeight()-1, 20, 20);

g2.dispose();

super.paintComponent(g);

}

};

p.setOpaque(false);

p.setBounds(x, y, w, h);

return p;

}

private JTextField makeField() {

    JTextField tf = new JTextField() {

        @Override protected void paintComponent(Graphics g) {

            Graphics2D g2 = (Graphics2D) g.create();

            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

            g2.setColor(new Color(255,255,255,30));

            g2.fillRoundRect(0, 0, getWidth(), getHeight(), 10, 10);

            g2.dispose();

            super.paintComponent(g);

        }

    };

    tf.setOpaque(false);

    tf.setForeground(Color.WHITE);

    tf.setCaretColor(ACCENT);

    tf.setFont(new Font("Segoe UI", Font.PLAIN, 14));
```

```
tf.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createLineBorder(new Color(255,255,255,60), 1, true),
    BorderFactory.createEmptyBorder(4, 10, 4, 10)));
return tf;
}

private void styleCombo(JComboBox<String> cb) {
    cb.setBackground(new Color(20, 50, 100));
    cb.setForeground(Color.RED);
    cb.setFont(new Font("Segoe UI", Font.PLAIN, 13));
    cb.setBorder(BorderFactory.createLineBorder(new Color(255,255,255,60), 1));
}

private void addFormRow(JPanel p, String label, JTextField field, int x, int y) {
    if (!label.isEmpty()) {
        JLabel lbl = new JLabel(label);
        lbl.setFont(new Font("Segoe UI", Font.BOLD, 13));
        lbl.setForeground(new Color(180, 215, 255));
        lbl.setBounds(x, y, 160, 28);
        p.add(lbl);
    }
    if (field != null) {
        field.setBounds(185, y, 270, 36);
        p.add(field);
    }
}

private JButton addBtn(JPanel p, String text, Color base, Color hover, int x, int y) {
    JButton btn = new JButton(text, base, hover);
    btn.setBounds(x, y, 200, 42);
    btn.addActionListener(this);
    p.add(btn);
    return btn;
}
```

```
}

private void printWelcome() {

    outputArea.setText(

        "┌───────────────────────────────────────────────────────────────────────────────────┐\n" +
        "│ Welcome to Bank System  │\n" +
        "└───────────────────────────────────────────────────────────────────────────────────┘\n\n" +
        " Use the left panel to enter\n" +
        " Customer ID, Account Number,\n" +
        " Account Type and Amount.\n\n" +
        " Then click any operation button.\n\n" +
        " Ready. ✓"

    );
}

@Override

public void actionPerformed(ActionEvent e) {

    String customerId = customerField.getText().trim();

    String accountNo = accountField.getText().trim();

    String accountType = (String) accountTypeBox.getSelectedItem();

    double amount = 0;

    try {

        if (!amountField.getText().trim().isEmpty())

            amount = Double.parseDouble(amountField.getText().trim());

    } catch (Exception ex) {

        outputArea.setText("⚠ Please enter a valid numeric amount!");

        return;

    }

    String now = sdf.format(new Date());

    if (e.getSource() == loginBtn) {

        outputArea.setText("✅ Login Successful\n\n" +
```

```
        "Welcome back!\nDate: " + now);
    }
    else if (e.getSource() == addCustomerBtn) {
        if (customerId.isEmpty()) { outputArea.setText("\u25b6 Enter Customer ID"); return; }
        if (customers.containsKey(customerId)) {
            outputArea.setText("\u25b6 Customer ID already exists");
            return;
        }
        customers.put(customerId, new Customer(customerId, "Customer-" + customerId,
            "Address", "9999999999", "123456789012"));
        outputArea.setText("\u2705 Customer Added Successfully\n\nID: " + customerId);
    }
    else if (e.getSource() == createAccountBtn) {
        if (customerId.isEmpty() || accountNo.isEmpty()) {
            outputArea.setText("\u25b6 Enter Customer ID and Account Number"); return;
        }
        if (accountType.equals("Select Account Type")) {
            outputArea.setText("\u25b6 Select an Account Type"); return;
        }
        if (amount < 0) { outputArea.setText("\u25b6 Opening balance cannot be negative");
return; }
        if (accounts.containsKey(accountNo)) {
            outputArea.setText("\u25b6 Account Number already exists"); return;
        }
        accounts.put(accountNo, new Account(accountNo, customerId, accountType,
amount));
        outputArea.setText("\u2705 Account Created Successfully\n\n" +
            "Account No : " + accountNo + "\n" +
            "Customer ID : " + customerId + "\n" +
            "Type      : " + accountType + "\n" +
```

```
"Opening Bal : ₹" + String.format("%.2f", amount));
}
else if (e.getSource() == depositBtn) {
    if (amount <= 0) { outputArea.setText("⚠ Enter a valid deposit amount"); return; }
    Account acc = accounts.get(accountNo);
    if (acc == null) { outputArea.setText("⚠ Account not found"); return; }
    acc.deposit(amount);
    transactions.add(new Transaction(accountNo, "Credit", amount, now));
    outputArea.setText("✅ Deposit Successful\n\n" +
        "Account : " + accountNo + "\n" +
        "Credited : ₹" + String.format("%.2f", amount) + "\n" +
        "Balance : ₹" + String.format("%.2f", acc.balance) + "\n" +
        "Date : " + now);
} else if (e.getSource() == withdrawBtn) {
    if (amount <= 0) { outputArea.setText("⚠ Enter a valid withdrawal amount"); return; }
    Account acc = accounts.get(accountNo);
    if (acc == null) { outputArea.setText("⚠ Account not found"); return; }
    if (!acc.withdraw(amount)) { outputArea.setText("❌ Insufficient Balance"); return; }
    transactions.add(new Transaction(accountNo, "Debit", amount, now));
    outputArea.setText("✅ Withdrawal Successful\n\n" +
        "Account : " + accountNo + "\n" +
        "Debited : ₹" + String.format("%.2f", amount) + "\n" +
        "Balance : ₹" + String.format("%.2f", acc.balance) + "\n" +
        "Date : " + now);
} else if (e.getSource() == transferBtn) {
    if (amount <= 0) { outputArea.setText("⚠ Enter a valid transfer amount"); return; }
    Account src = accounts.get(accountNo);
```

```
        if (src == null) { outputArea.setText("⚠ Source account not found"); return; }

        String destNo = JOptionPane.showInputDialog(this, "Enter Destination Account Number:");

        if (destNo == null || destNo.trim().isEmpty()) {
            outputArea.setText("Transfer cancelled"); return;
        }

        destNo = destNo.trim();

        Account dest = accounts.get(destNo);

        if (dest == null) { outputArea.setText("⚠ Destination account not found"); return; }

        if (destNo.equals(accountNo)) { outputArea.setText("⚠ Cannot transfer to same account"); return; }

        if (!src.withdraw(amount)) { outputArea.setText("❌ Insufficient Balance in source account"); return; }

        dest.deposit(amount);

        transactions.add(new Transaction(accountNo, "Transfer-Out", amount, now));
        transactions.add(new Transaction(destNo, "Transfer-In", amount, now));

        outputArea.setText("✅ Transfer Successful\n\n" +
            "From   : " + accountNo + "\n" +
            "To     : " + destNo + "\n" +
            "Amount : ₹" + String.format("%.2f", amount) + "\n" +
            "Date   : " + now + "\n\n" +
            "Your new balance: ₹" + String.format("%.2f", src.balance));
    }

    else if (e.getSource() == balanceBtn) {
        Account acc = accounts.get(accountNo);

        if (acc == null) { outputArea.setText("⚠ Account not found"); return; }

        outputArea.setText("🏠 Account Balance\n\n" +
            "Account : " + accountNo + "\n" +
            "Type    : " + acc.accountType + "\n" +
            "Balance : ₹" + String.format("%.2f", acc.balance));
    }
```



```
}  
else if (e.getSource() == transactionBtn) {  
    StringBuilder sb = new StringBuilder(" 📄 Transaction History\n");  
    sb.append("Account : ").append(accountNo).append("\n");  
    sb.append("—".repeat(38)).append("\n");  
    boolean any = false;  
    for (Transaction t : transactions) {  
        if (t.accountNumber.equals(accountNo)) {  
            sb.append(String.format("%-14s ₹%-10.2f\n", t.type, t.amount));  
            sb.append(" ").append(t.date).append("\n");  
            sb.append("·".repeat(38)).append("\n");  
            any = true;  
        }  
    }  
    if (!any) sb.append("No transactions found.");  
    outputArea.setText(sb.toString());  
}  
else if (e.getSource() == miniStatementBtn) {  
    Account acc = accounts.get(accountNo);  
    if (acc == null) { outputArea.setText("⚠ Account not found"); return; }  
    StringBuilder sb = new StringBuilder();  
    sb.append(" 📄 MINI STATEMENT\n");  
    sb.append("=".repeat(36)).append("\n");  
    sb.append("Acct No : ").append(accountNo).append("\n");  
    sb.append("Type : ").append(acc.accountType).append("\n");  
    sb.append("Balance : ₹").append(String.format("%.2f", acc.balance)).append("\n");  
    sb.append("—".repeat(36)).append("\n");  
    sb.append("Last 5 Transactions:\n\n");  
    int count = 0;
```

```
for (int i = transactions.size() - 1; i >= 0; i--) {
    Transaction t = transactions.get(i);
    if (t.accountNumber.equals(accountNo)) {
        sb.append(String.format("%-14s ₹%.2f\n", t.type, t.amount));
        sb.append(" ").append(t.date).append("\n\n");
        if (++count == 5) break;
    }
} if (count == 0) sb.append("No transactions found.");
outputArea.setText(sb.toString());
} else if (e.getSource() == loanBtn) {
    if (accountNo.isEmpty()) { outputArea.setText("⚠ Enter Account Number"); return;
}

Account acc = accounts.get(accountNo);
if (acc == null) { outputArea.setText("⚠ Account not found"); return; }

String loanInput = JOptionPane.showInputDialog(this, "Enter Loan Amount (₹):");
if (loanInput == null || loanInput.trim().isEmpty()) {
    outputArea.setText("Loan request cancelled"); return;
}

double loanAmount;
try { loanAmount = Double.parseDouble(loanInput.trim()); }
catch (Exception ex) { outputArea.setText("⚠ Invalid loan amount"); return; }

if (loanAmount <= 0) { outputArea.setText("⚠ Loan amount must be positive");
return; }

String monthsInput = JOptionPane.showInputDialog(this, "Enter Repayment Period
(months, e.g. 12):");

if (monthsInput == null || monthsInput.trim().isEmpty()) {
    outputArea.setText("Loan request cancelled"); return;
}

int months;
```

```
try { months = Integer.parseInt(monthsInput.trim()); }  
catch (Exception ex) { outputArea.setText("⚠ Invalid months value"); return; }  
if (months <= 0) { outputArea.setText("⚠ Months must be a positive integer");  
return; }
```

```
Loan loan = new Loan(UUID.randomUUID().toString().substring(0,  
8).toUpperCase(),
```

```
accountNo, loanAmount, months);
```

```
loans.add(loan);
```

```
outputArea.setText("✅ Loan Applied Successfully\n\n" +
```

```
"Loan ID : " + loan.loanId + "\n" +
```

```
"Amount : ₹" + String.format("%.2f", loan.amount) + "\n" +
```

```
"Months : " + loan.months + "\n" +
```

```
"Monthly EMI: ₹" + String.format("%.2f", loan.emi) + "\n" +
```

```
"Status : " + loan.status);
```

```
}
```

```
else if (e.getSource() == emiBtn) {
```

```
Account acc = accounts.get(accountNo);
```

```
if (acc == null) { outputArea.setText("⚠ Account not found"); return; }
```

```
boolean found = false;
```

```
for (Loan loan : loans) {
```

```
if (loan.accountNumber.equals(accountNo) &&  
!loan.status.equals("Completed")) {
```

```
if (!loan.payEMI(acc)) {
```

```
outputArea.setText("❌ Insufficient Balance to pay EMI of ₹" +
```

```
String.format("%.2f", loan.emi));
```

```
return;
```

```
} transactions.add(new Transaction(accountNo, "EMI Payment", loan.emi,  
now));
```

```
found = true;
```

```

outputArea.setText("✅ EMI Paid Successfully\n\n" +
    "Loan ID : " + loan.loanId + "\n" +
    "EMI Paid : ₹" + String.format("%.2f", loan.emi) + "\n" +
    "Total Paid: ₹" + String.format("%.2f", loan.paidAmount) + "\n" +
    "Remaining : ₹" + String.format("%.2f", loan.amount - loan.paidAmount) +
"\n" +
    "Acct Bal : ₹" + String.format("%.2f", acc.balance) + "\n" +
    "Status : " + loan.status);

Break;}

} if (!found) outputArea.setText("⚠ No active loan found for account: " +
accountNo);}

else if (e.getSource() == reportBtn) {
    double totalDeposits = 0, totalWithdrawals = 0;
    for (Transaction t : transactions) {
        if (t.type.equals("Credit")) totalDeposits += t.amount;
        if (t.type.equals("Debit")) totalWithdrawals += t.amount;}
    int activeLoans = 0;
    for (Loan l : loans) if (l.status.equals("Approved")) activeLoans++;
    outputArea.setText("📊 SYSTEM REPORT\n" +
        "=".repeat(34) + "\n" +
        "Total Customers : " + customers.size() + "\n" +
        "Total Accounts : " + accounts.size() + "\n" +
        "Total Transactions: " + transactions.size() + "\n" +
        "Active Loans : " + activeLoans + "\n" +
        "Total Loans : " + loans.size() + "\n" +
        "—".repeat(34) + "\n" +
        "Total Deposited : ₹" + String.format("%.2f", totalDeposits) + "\n" +
        "Total Withdrawn : ₹" + String.format("%.2f", totalWithdrawals));}

else if (e.getSource() == interestBtn) {

```

```
Account acc = accounts.get(accountNo);

if (acc == null) { outputArea.setText("⚠ Account not found"); return; }

double before = acc.balance;

acc.addInterest();

double interest = acc.balance - before;

transactions.add(new Transaction(accountNo, "Interest", interest, now));

outputArea.setText("✅ Interest Added (4%)\n\n" +

    "Previous Bal : ₹" + String.format("%.2f", before) + "\n" +

    "Interest      : ₹" + String.format("%.2f", interest) + "\n" +

    "Updated Bal  : ₹" + String.format("%.2f", acc.balance));}

else if (e.getSource() == clearBtn) {

    customerField.setText("");

    accountField.setText("");

    amountField.setText("");

    accountTypeBox.setSelectedIndex(0);

    printWelcome();

}} public static void main(String[] args) {

    SwingUtilities.invokeLater(() -> {

        try {

            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName()); }

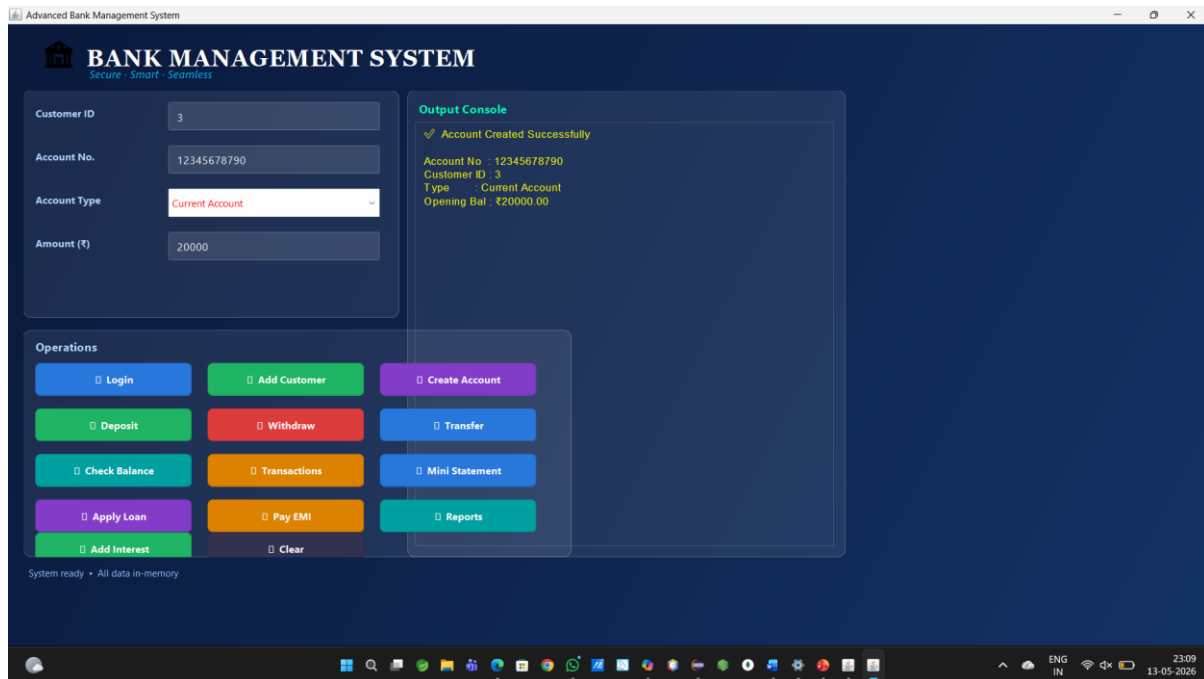
            catch (Exception ignored) {}

            new BankUI();

        });}

    }
```

CHAPTER-5

OUTPUT

CHAPTER-6**CONCLUSION**

The Advanced Java Banking Management System provides a complete solution to the challenges of manual banking by integrating customer, account, transaction, and loan management into a single application. Built with object-oriented design principles and a Swing-based graphical interface, it ensures automation, accuracy, and user-friendliness. The system validates inputs, prevents errors, and updates records in real time, thereby improving efficiency and reliability. Features such as deposits, withdrawals, transfers, loan applications, EMI deductions, interest calculation, and transaction reporting make it a robust prototype of modern banking workflows.

In conclusion, this project demonstrates how Advanced Java technologies (Swing, event handling, collections, and OOP) can be effectively applied to create a secure, scalable, and maintainable banking solution. It not only serves as an academic project but also as a foundation for real-world banking applications, showcasing the power of Java in building enterprise-level financial systems.

REFERENCES

- ☐ Oracle Java SE Swing Documentation
- ☐ Oracle Java SE AWT Documentation
- ☐ Oracle Java 2D Graphics Tutorial
- ☐ Oracle Pluggable Look and Feel Architecture Guidelines
- ☐ W3C Compositing and Blending Specification
- ☐ Oracle Java SE Collections Framework Documentation
- ☐ IEEE 754 Standard for Floating-Point Arithmetic